GCC Web Application Deployment Plan

Executive Summary

This document outlines the comprehensive deployment strategy for the GCC Web Application from local development to production environments using CI/CD pipelines and Azure cloud services.

Table of Contents

1. Architecture Overview
2. Environment Strategy
3. CI/CD Pipeline
4. Deployment Process
5. Database Migration Strategy
6. Security Considerations
7. Monitoring and Rollback
8. Quick Start Guide
9. Troubleshooting
10. Support and Contacts
11. Appendix

Architecture Overview

Technology Stack

- **Framework**: ASP.NET Core 8.0
- **Database**: SQL Server / SQLite
- **Cloud Provider**: Microsoft Azure
- **CI/CD**: GitHub Actions / Azure DevOps
- **Container**: Docker (optional)
- **Monitoring**: Application Insights

Infrastructure Components

```
Local Development
   ↓
Source Control (GitHub/Azure Repos)
   ↓
CI/CD Pipeline
   ↓
Azure App Service
   ├── Development Slot
   ├── Staging Slot
   └── Production (Blue/Green)
```

Environment Strategy

1. Local Development

- **Purpose**: Developer workstations
- **Database**: SQLite (gcc_database.db)
- **Configuration**: appsettings.Development.json
- **URL**: http://localhost:5000

2. Development Environment

- **Purpose**: Integration testing, feature development
- **Database**: Azure SQL Database (Basic tier)
- **Configuration**: appsettings.Development.json
- **URL**: https://gcc-webapp-dev.azurewebsites.net
- **Deployment**: Automatic on push to `develop` branch

3. Staging Environment

- **Purpose**: UAT, pre-production testing
- **Database**: Azure SQL Database (Standard tier)
- **Configuration**: appsettings.Staging.json
- **URL**: https://gcc-webapp-staging.azurewebsites.net
- **Deployment**: Automatic on push to `main` branch

4. Production Environment

- **Purpose**: Live application

- **Database**: Azure SQL Database (Standard/Premium tier)
- **Configuration**: appsettings.Production.json
- **URL**: https://gcc-webapp.com
- **Deployment**: Manual approval after staging

CI/CD Pipeline

GitHub Actions Workflow

Location: `.github/workflows/ci-cd-pipeline.yml`

Pipeline Stages:

1. Build & Test

- Restore NuGet packages
- Build solution
- Run unit tests
- Code coverage analysis

2. Security Scan

- OWASP dependency check
- Security vulnerability scanning
- Code quality analysis

3. Deploy to Development

- Triggered by: Push to `develop`
- Automatic deployment
- Database migrations
- Health checks

4. Deploy to Staging

- Triggered by: Push to `main`
- Automatic deployment
- Smoke tests
- Performance tests

5. Deploy to Production

- Triggered by: Manual approval
- Blue-Green deployment
- Zero-downtime deployment
- Automatic rollback on failure

Azure DevOps Pipeline

Location: `azure-pipelines.yml`

Alternative pipeline configuration for organizations using Azure DevOps.

Deployment Process

Prerequisites

1. Azure Resources

```bash
# Create Resource Groups
az group create --name gcc-dev-rg --location eastus
az group create --name gcc-staging-rg --location eastus
az group create --name gcc-prod-rg --location eastus

# Create App Service Plans
az appservice plan create --name gcc-dev-plan --resource-group gcc-dev-rg --sku B1
az appservice plan create --name gcc-staging-plan --resource-group gcc-staging-rg --sku S1
az appservice plan create --name gcc-prod-plan --resource-group gcc-prod-rg --sku S2

# Create Web Apps
az webapp create --name gcc-webapp-dev --resource-group gcc-dev-rg --plan gcc-dev-plan
az webapp create --name gcc-webapp-staging --resource-group gcc-staging-rg --plan gcc-staging-plan
az webapp create --name gcc-webapp-prod --resource-group gcc-prod-rg --plan gcc-prod-plan
```

2. Database Setup

```bash
```

```
# Create SQL Servers
az sql server create --name gcc-sql-dev --resource-group gcc-dev-rg --admin-user gccadmin --admin-password <pass
az sql server create --name gcc-sql-staging --resource-group gcc-staging-rg --admin-user gccadmin --admin-passwor
az sql server create --name gcc-sql-prod --resource-group gcc-prod-rg --admin-user gccadmin --admin-password <p

# Create Databases
az sql db create --name GCCDatabase --server gcc-sql-dev --resource-group gcc-dev-rg --edition Basic
az sql db create --name GCCDatabase --server gcc-sql-staging --resource-group gcc-staging-rg --edition Standard
az sql db create --name GCCDatabase --server gcc-sql-prod --resource-group gcc-prod-rg --edition Standard
```

3. Key Vault Setup

bash

```
# Create Key Vaults
az keyvault create --name gcc-kv-dev --resource-group gcc-dev-rg
az keyvault create --name gcc-kv-staging --resource-group gcc-staging-rg
az keyvault create --name gcc-kv-prod --resource-group gcc-prod-rg

# Store secrets
az keyvault secret set --vault-name gcc-kv-dev --name "ConnectionString" --value "<connection-string>"
az keyvault secret set --vault-name gcc-kv-dev --name "JwtKey" --value "<jwt-key>"
```

Manual Deployment Steps

Deploy from Local to Development:

powershell

```
# Use the provided PowerShell script
.\Scripts\deploy-local-to-dev.ps1 `
    -Environment "Development" `
    -ResourceGroup "gcc-dev-rg" `
    -AppName "gcc-webapp-dev" `
    -ConnectionString "<your-connection-string>" `
    -KeyVaultName "gcc-kv-dev"
```

Deploy using Azure CLI:

bash

```
# Build and publish
dotnet publish -c Release -o ./publish

# Create deployment package
cd publish
zip -r ../deploy.zip .
cd ..

# Deploy to Azure
az webapp deploy --resource-group gcc-dev-rg --name gcc-webapp-dev --src-path deploy.zip --type zip
```

Database Migration Strategy

Migration Approach

1. **Version Control**: All migrations tracked in source control
2. **Automated Migrations**: Applied during deployment
3. **Rollback Strategy**: Maintain migration history for reversal
4. **Data Backup**: Automatic backup before migrations

Running Migrations

Local Development:

```
bash

# Add new migration
dotnet ef migrations add <MigrationName>

# Update database
dotnet ef database update
```

Deployment Pipeline:

```
powershell
```

```
# Use the migration script
.\Scripts\database-migration.ps1 `
    -Environment "Development" `
    -Provider "SqlServer" `
    -ConnectionString "<connection-string>" `
    -CreateBackup
```

Migration Best Practices

1. Always test migrations in development first
2. Create backups before production migrations
3. Use idempotent scripts for safety
4. Monitor migration execution time
5. Have rollback scripts ready

Security Considerations

Secrets Management

- **Development**: User secrets / appsettings.Development.json
- **Production**: Azure Key Vault
- **CI/CD**: GitHub Secrets / Azure DevOps Variables

Required Secrets:

yaml

```
# GitHub Secrets
AZURE_CREDENTIALS_DEV
AZURE_CREDENTIALS_STAGING
AZURE_CREDENTIALS_PROD
DEV_CONNECTION_STRING
STAGING_CONNECTION_STRING
PROD_CONNECTION_STRING
```

Security Checklist:

- ☐ Enable HTTPS only
- ☐ Configure CORS properly
- ☐ Enable managed identity

- ☐ Implement rate limiting
- ☐ Enable Application Insights
- ☐ Configure firewall rules
- ☐ Enable SQL Database auditing
- ☐ Implement data encryption
- ☐ Regular security scans
- ☐ Vulnerability assessments

Monitoring and Rollback

Health Checks

Endpoint: `/health`

```csharp
// Health check implementation
app.MapHealthChecks("/health", new HealthCheckOptions
{
    ResponseWriter = UIResponseWriter.WriteHealthCheckUIResponse
});
```

Monitoring Setup

1. Application Insights

- Performance monitoring
- Error tracking
- Custom metrics
- User analytics

2. Azure Monitor

- Resource metrics
- Alerts configuration
- Log analytics

3. Alerts Configuration

```bash
```

```
# Create alert for high CPU
az monitor metrics alert create \
  --name high-cpu-alert \
  --resource-group gcc-prod-rg \
  --scopes /subscriptions/{sub-id}/resourceGroups/gcc-prod-rg/providers/Microsoft.Web/sites/gcc-webapp-prod \
  --condition "avg Percentage CPU > 80" \
  --window-size 5m \
  --evaluation-frequency 1m
```

Rollback Procedures

Immediate Rollback (Blue-Green):

```
bash

# Swap slots back
az webapp deployment slot swap \
  --name gcc-webapp-prod \
  --resource-group gcc-prod-rg \
  --slot production \
  --target-slot blue
```

Database Rollback:

```
bash

# Revert last migration
dotnet ef database update <PreviousMigration>

# Or restore from backup
az sql db restore \
  --dest-name GCCDatabase \
  --name GCCDatabase \
  --resource-group gcc-prod-rg \
  --server gcc-sql-prod \
  --time "2024-01-20T03:00:00Z"
```

Quick Start Guide

First-Time Setup

1. Clone Repository

```bash
git clone https://github.com/your-org/gcc-webapp.git
cd gcc-webapp
```

2. Install Prerequisites

- .NET 8.0 SDK
- Azure CLI
- Visual Studio 2022 / VS Code

3. Configure Local Environment

```bash
# Set user secrets
dotnet user-secrets set "ConnectionStrings:DefaultConnection" "<your-connection>"
dotnet user-secrets set "Jwt:Key" "<your-jwt-key>"
```

4. Run Locally

```bash
dotnet restore
dotnet build
dotnet ef database update
dotnet run
```

Deployment Checklist

Pre-Deployment:

- ☐ All tests passing
- ☐ Code review completed
- ☐ Security scan clean
- ☐ Database backup created
- ☐ Release notes prepared

Deployment:

- ☐ Deploy to development

- ☐ Run smoke tests
- ☐ Deploy to staging
- ☐ UAT sign-off
- ☐ Deploy to production
- ☐ Verify health checks

Post-Deployment:

- ☐ Monitor application logs
- ☐ Check performance metrics
- ☐ Verify all features
- ☐ Update documentation
- ☐ Notify stakeholders

Troubleshooting

Common Issues

1. Deployment Fails

```bash
# Check deployment logs
az webapp log tail --name gcc-webapp-dev --resource-group gcc-dev-rg

# Check deployment status
az webapp deployment list --name gcc-webapp-dev --resource-group gcc-dev-rg
```

2. Database Connection Issues

```bash
# Test connection
az sql db show-connection-string --server gcc-sql-dev --name GCCDatabase --client ado.net

# Check firewall rules
az sql server firewall-rule list --server gcc-sql-dev --resource-group gcc-dev-rg
```

3. Application Errors

```bash
# Enable detailed errors
az webapp config appsettings set --name gcc-webapp-dev --resource-group gcc-dev-rg --settings ASPNETCORE_DETA

# View live logs
az webapp log tail --name gcc-webapp-dev --resource-group gcc-dev-rg
```

Support and Contacts

- **DevOps Team**: devops@yourcompany.com
- **Security Team**: security@yourcompany.com
- **Database Admin**: dba@yourcompany.com
- **On-Call Support**: +1-XXX-XXX-XXXX

Appendix

Useful Commands

```bash
# View app settings
az webapp config appsettings list --name gcc-webapp-dev --resource-group gcc-dev-rg

# Restart app
az webapp restart --name gcc-webapp-dev --resource-group gcc-dev-rg

# Scale up
az appservice plan update --name gcc-dev-plan --resource-group gcc-dev-rg --sku S1

# Enable always on
az webapp config set --name gcc-webapp-dev --resource-group gcc-dev-rg --always-on true
```

Environment Variables

```yaml
```

ASPNETCORE_ENVIRONMENT: Development|Staging|Production
ApplicationInsights__ConnectionString: <app-insights-connection>
KeyVault__VaultUri: https://<vault-name>.vault.azure.net/
WEBSITE_TIME_ZONE: Eastern Standard Time

Document Information

- **Document Version**: 1.0
- **Last Updated**: August 2025
- **Next Review**: September 2025
- **Document Owner**: DevOps Team
- **Classification**: Internal Use Only